Capítulo 4: Álgebra de tablas y su implementación

Parte 1

Tabla de contenidos

- Bloque 1: Visión general 3
- Bloque 2: Tablas y esquemas 7
- Bloque 3: Proyección 9
- Bloque 4: Proyección generalizada 11
- Bloque 5: Operador físico para proyección 15
- Bloque 6: Selección 16
- Bloque 7: Operadores físicos para selección 20
- Bloque 8: Producto cartesiano 31
- Relación entre SQL y álgebra de tablas. 38

Bloque 1: Visión general

- **Para cada operador del álgebra de tablas vamos a estudiar:**
  - Su definición
  - Implementación del operador de usando operadores físicos
    - Aquí se incluye la estimación del costo de ejecutar el operador físico
  - Estimación de tamaño de almacenamiento del resultado de evaluar un operador físico.

Bloque 1: Visión general

- Hace falta medir el costo de un operador del álgebra de tablas, y también medir el costo de una consulta del álgebra de tablas.
- Para ellos se contará el número de **transferencia de bloques** de disco.
  - Por simplicidad de las cuentas vamos a asumir que todas las transferencias de bloques tienen el mismo costo.
  - No distinguiremos entre las transferencias de bloques de lectura y las de escritura (a pesar de que se demora más en escribir un bloque que leerlo de disco)

Bloque 1: Visión general

- Además, se puede contar la cantidad de **accesos a bloques**:
  - El tiempo de un acceso a bloque es el tiempo que le lleva a la cabeza lectora posicionarse en un bloque deseado.
- **Para discos rígidos:**
  - El tiempo de un acceso a bloque suele ser de alrededor de 4 mseg.
  - El tiempo de transferencia de bloques en suele ser de 0,1 mseg, asumiendo un tamaño de bloque de 4 KiB y una tasa de transferencia de 40 MB por segundo.
- **Para disco de estado sólido (SSD):**
  - El acceso a bloque puede llevar 35 microsegundos, gracias al acceso electrónico de la memoria flash.
  - Para SSD de 2 Gbps y tamaño de bloque de 4096 B una transferencia de bloque llevaría 16 microsegundos.
- **Conclusión**: con SSD no hay tanta diferencia entre los accesos a bloque y las transferencias bloque. En cambio en un disco rígido la diferencia es enorme.

Bloque 1: Visión general

- Los costos de todos los algoritmos físicos dependen del **tamaño del búfer** en memoria principal.
  - En el mejor caso todos los datos pueden ser leídos en búfer y el disco no necesita ser accedido de nuevo.
  - En el peor caso asumimos que el búfer puede sostener solo unos pocos bloques de datos – aproximadamente un bloque por tabla.
- Cuando presentamos estimaciones de costos, asumimos generalmente el peor caso.
- Vamos a ignorar los costos de CPU por simplicidad.

Bloque 2: Tablas y esquemas

- Vieron que un esquema relacional es una lista de nombres de atributos y usaron la notación.
- R = (A1,…, An)
- Una forma más refinada de dar un esquema que adoptamos es:
  - R = (A1::T1, …, An:: Tn)
  - Ti es tipo para valores de Ai
  - Dom(R) = T1 x…x Tn
- Vieron que r(A1,…,An) significa que la tabla r tiene esquema (A1,…,An).
- Para dar más detalle ponemos: r(A1::T1, …, An:: Tn)

$nombre$			
$campo_1 :: \tau_1$	$campo_2 :: \tau_2$	$\ldots$	$campo_N :: \tau_N$
v_{11}	v_{12}	$\ldots$	v_{1N}
v_{21}	v_{22}	$\ldots$	v_{2N}
$\vdots$	$\vdots$	$\ddots$	$\vdots$
v_{M1}	v_{M2}	$\ldots$	v_{MN}

Bloque 2: Tablas y esquemas

- **Ejemplo**: Equemas para: biblioteca, bibliotecario, trabajaEn

  biblioteca(nombreBib:: string, calle::string, número: integer)

  bibliotecario(dni:: integer, antigüedad: integer)

  trabajaEn(dni::integer, nombreBib::string)

- **Ejemplo**: tuplas de las tablas anteriores son:

  (“FaMAF”, “Medina Allende”, 0) ∈ biblioteca

  (1232, 5) ∈ bibliotecario

  (1232, “FaMAF”) ∈ trabajaEn

- **Ejemplo**: biblioteca = [(“FaMAF”, “Medina Allende”, 0)]

Bloque 3: Proyección

- Relación *r*

A	*B*	*C*
α	10	1
α	20	1
β	30	1
β	40	2

- Selecciona A and C
  - proyección
  - $\Pi_{A, C}$ (r)

A	*C*
α	1
α	1
β	1
β	2

¿Cómo definir recursivamente la operación que acabamos de hacer?

Bloque 3: Proyección

- Relación *r*

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

- Selecciona A and C
  - proyección
  - $\Pi_{A,C}(r)$

A	C
α	1
α	1
β	1
β	2

$\Pi_{A,C}[] = []$

$\Pi_{A,C}(t{:}r) = (t.A, t.C) : (\Pi_{A,C}\ r)$

$= ((\backslash t' \rightarrow (t'.A, t'.C))\ t) : (\Pi_{A,C}\ r)$

$\Pi_{A,C} = map\ (\backslash t' \rightarrow (t'.A, t'.C))$

Bloque 4: Proyección Generalizada

profe			
legajo	nombres	apellidos	sueldo
p1	"Benjamin"	"Pierce"	3000
p2	"Patricia"	"Selinger"	6000
p3	"Edgar F"	"Codd"	5500
p4	"Barbara"	"Liskov"	5600

Hacer la consulta: obtener los sueldos anuales de los profesores.

sueldos_anuales	
legajo	_
p1	39000
p2	78000
p3	71500
p4	72800

¿Esto es una proyección?

Bloque 4: Proyección Generalizada

profe			
legajo	nombres	apellidos	sueldo
p1	"Benjamin"	"Pierce"	3000
p2	"Patricia"	"Selinger"	6000
p3	"Edgar F"	"Codd"	5500
p4	"Barbara"	"Liskov"	5600

Hacer la consulta: obtener los sueldos anuales de los profesores.

sueldos_anuales	
legajo	_
p1	39000
p2	78000
p3	71500
p4	72800

- **No** porque la columna de 'sueldos anuales' no es atributo de la tabla.
- Pero se parece mucho a proyección.
- Por eso **generalizaremos** la proyección para capturar situaciones como esta.

Bloque 4: Proyección Generalizada

profe			
legajo	nombres	apellidos	sueldo
p1	"Benjamin"	"Pierce"	3000
p2	"Patricia"	"Selinger"	6000
p3	"Edgar F"	"Codd"	5500
p4	"Barbara"	"Liskov"	5600

Hacer la consulta: obtener los sueldos anuales de los profesores.

sueldos_anuales	
legajo	_
p1	39000
p2	78000
p3	71500
p4	72800

$$\text{sueldos_anuales} = \Pi_{\text{legajo},\ \text{sueldo}*13}(\text{profe})$$

legajo = (\t -> t.legajo)

sueldo * 13 = (\t -> t.sueldo * 13)

Bloque 4: Proyección Generalizada

- Generalizando:

  $\Pi_{f1,\ldots,fn}\ [] = []$

  $\Pi_{f1,\ldots,fn}\ (t:r) = (f1(t), \ldots, fn(t)) : \Pi_{f1,\ldots,fn}\ (r) = ((\backslash t' \rightarrow (f1(t'),\ldots,fn(t'))\ t) : \Pi_{f1,\ldots,fn}\ (r)$

- O sea que tiene la forma de un map.
- $\Pi_{f1,\ldots,fn} = map\ (\backslash t' \rightarrow (f1(t'),\ldots,fn(t')))$

Bloque 5: Operador físico para proyección

```
Proyección (r, R, AL, f) // AL attribute list
  if not  exist(f): then create(f, AL) // resultado en archivo
  Buffer_salida = vacío
  B = leer_primer_bloque(r)
  While B ≠ null do
      For t in B do
          t' = Project (t, R, AL)
          if no_hay_lugar(buffer_salida)
             write(buffer_salida, f)
             buffer_salida = vacio
           fi
          insertar(t', buffer_salida)
        od
       B = Leer_siguiente_bloque(r)
  od
 if buffer_salida ≠ vacío then write(buffer_salida, f) fi
```

- Estimación de costo:
  - Asumimos que la tabla está organizada secuencialmente y todos sus bloques están contiguos en disco.
- Hay b_r transferencias de bloques + b_f transferencias de bloques.
- Además hay 2* b_f accesos a bloques.
- Si no hubiera manejo de f, y se imprime el resultado en pantalla,
  - solo habría 1 acceso a bloques y b_r transferencias de bloques.
- La proyección generalizada puede ser implementada de la misma manera que la proyección.

Bloque 6: Selección

- Sean las tablas:

profe			
legajo	nombres	apellidos	sueldo
p1	"Benjamin"	"Pierce"	3000
p2	"Patricia"	"Selinger"	6000
p3	"Edgar F"	"Codd"	5500
p4	"Barbara"	"Liskov"	5600

curso		
id	legajo	nombre
c1	p2	"Optimización de Consultas"
c2	p3	"Fundamentos de BD"
c3	p2	"Análisis de Datos"
c4	p1	"Fundamentos del Software"
c5	p4	"Programación OO"

- Sea la consulta: obtener los cursos enseñados por Patricia

cursos_patricia		
id	legajo	nombre
c1	p2	"Optimización de Consultas"
c3	p2	"Análisis de Datos"

$$\text{cursos_patricia} = \sigma_{\text{legajo}=\text{p2}}(\text{curso})$$

Bloque 6: Selección

- Vamos a aplicar la selección a predicados.
- Los **predicados** son funciones booleanas.
- En el ejemplo anterior:
- Legajo == p2 =def (\t -> t.legajo == p2)
- Esto se puede generalizar más aun:
- A rel v =def (\t -> t.A rel v) donde A atributo y v valor constante.
- A1 rel A2 = def (\t -> t.A1 rel t.A2), donde A1 y A2 atributos.
- Donde rel ∈ {<, >, ==, ….}
- Las anteriores son **predicados básicos o atómicos**.

Bloque 6: Selección

- Los predicados básicos se pueden combinar usando **conectivos booleanos.**
- El conjunto de conectivos que consideramos es:
- con ∈ {&&, !!, not}
- **Por ejemplo**:
- A > v && A1 == A2   = def (\t -> t.A > v && t.A1 == t.A2)
- **Ahora que sabemos qué son los predicados, ¿cómo se puede definir el operador de selección recursivamente?**

Bloque 6: Selección

- El operador de selección se define recursivamente como sigue:
- $\sigma_P\ [] = []$
- $\sigma_P\ (t{:}r) =$ if $p(t)$ then $t : (\sigma_P\ r)$ else $\sigma_P\ r$

Bloque 7: Operadores físicos para selección

```
SelecciónSecuencial(r, R, P)
  if not  exist(f): then create(f, R) // resultados van a archivo
  Buffer_salida = vacío
  B = leer_primer_bloque(r)
  While B ≠ null do
     For t in B do   // seleccionar resultados de bloque B
            if P(t) then
               if no_hay_lugar(buffer_salida)
                  write(buffer_salida,f)
                  buffer_salida = vacío
               fi
               insertar(t, buffer_salida) fi
        od
        B = Leer_siguiente_bloque(r)
  od
  if buffer_salida ≠ vacío then write(buffer_salida, f) fi
```

- **Algoritmo de búsqueda lineal**
  - Escanear cada bloque del archivo y testear todos los registros para ver si satisfacen la condición de selección.
  - **Estimación de costo**
    - Asumimos organización secuencial de r con bloques contiguos.
    - Hay b_r transferencias de bloques + b_f transferencias de bloques.
    - Además hay 2* b_f accesos a bloques.
    - Si no hubiera manejo de f, y se imprime el resultado en pantalla,
      - solo habría 1 acceso a bloques y b_r transferencias de bloques.

Bloque 7: Operadores físicos para selección

```
SelecciónIndPrimIgCC(r, R, A, c, f)
    if not  exist(f): then create(f, R) fi
   Buffer_salida = vacío
    P = buscar_índice(C)
    if P ≠ null then
       B = leer_bloque(r,P)
       For t in B do  // sacar resultado de bloque B
                if t[A] == c then
                   insertar(t, buffer_salida) fi
                   break
                fi
        od
        write(buffer_salida, f)
    fi
```

- **Algoritmo para índice primario usando árbol B+ con igualdad en clave candidata**
  - Recorrer la altura del árbol más una E/S para recoger el registro.
  - Cada una de estas operaciones requiere un acceso a bloque y una transferencia de bloque.
  - El costo es (h_i + 1) transferencias de bloque y (h_i + 1) accesos a bloque;
    - h_i denota la altura del índice.

Bloque 7: Operadores físicos para selección

- **Algoritmo para índice secundario usando árbol B+, igualdad en clave candidata**
  - Este caso es similar al índice primario y por lo tanto el costo es el mismo.
  - (h_i + 1) transferencias de bloque y (h_i + 1) accesos a bloque.
  - donde h_i denota la altura del índice.

Bloque 7: Operadores físicos para selección

- **Algoritmo para índice primario usando árbol B+ igualdad para no clave candidata**
  - Hay un acceso a bloque para cada nivel del árbol y un acceso a bloque para la tabla
  - **Estimación de costo:**
    - Asumir los registros de la tabla están en bloques consecutivos.
    - Sea *b* el número de bloques conteniendo registros con la clave de búsqueda especificada, todos los cuales son leídos.
      - Estos bloques son bloques hoja asumiendo que están almacenados secuencialmente y no requieren accesos adicionales a bloque.
    - El costo es h + 1 + b transferencia de bloques, h + 1 accesos a bloques:
      - h_i transferencias de bloque y h_i accesos a bloque para recorrer el árbol B+. Donde h_i es la altura del árbol B+.
      - Un acceso a bloque para buscar los registros en el archivo de la tabla.
      - Un total de b transferencias de bloque para acceder a los registros con la clave de búsqueda.
      - Asumimos que el resultado no se guarda en disco.

Bloque 7: Operadores físicos para selección

Algoritmo para índice secundario usando árbol B+ igualdad para no clave candidata

```
SelecciónIndPrimIgNoCC(r, R, A, C, f)
   if not exist(f) then create(f, R) fi  // arch results
   buffer_salida = vacío
   pb = buscar_indice(C)
   P = lista vacía
   while pb ≠ null do //saca punteros de buckets
       K = leer_bloque(buckets, pb)
       agregar a P todos los punteros de K
       pb = sig(K)
    od
    ordenar P por número de bloque
    bloque_actual = null
for p in P do // saca registros de punteros
   if bloque(p) ≠ bloque_actual then
       B = leer_bloque(r,bloque(p))
       bloque_actual = bloque(p) fi
    t = tupla de B en posición pos(p)
    if no_hay_lugar(buffer_salida, t) then
       write(buffer_salida, f)
        buffer_salida = vacío fi
    insertar(t, buffer_salida)
od
 if buffer_salida ≠ vacío then
    write(buffer_salida, f) fi
 return f
```

Bloque 7: Operadores físicos para selección

- **Algoritmo para índice secundario usando árbol B+, igualdad en no clave candidata**
  - Aquí el costo de recorrido del índice es el mismo.
  - Sin embargo, cada registro del resultado puede estar en un bloque diferente, requiriendo un acceso a bloque por registro.
  - Sea *n* el número de registros recogidos.
  - Sea *k* el número de bloques de los buckets
  - El costo es en el peor caso ($h_i + k + n$) transferencias de bloque y ($h_i + k + n$) accesos a bloques.
  - Si el resultado se escribe en disco, a esto hay que sumarle:
    - b_f transferencias de bloques y b_f accesos a bloques.
  - Entonces el costo total puede ser muy alto.

Bloque 7: Operadores físicos para selección Selecciones involucrando comparaciones

- Podemos implementar selecciones de la forma $\sigma_{A \leq V}(r)$ o $\sigma_{A \geq V}(r)$ usando:
  - un escaneo de archivo lineal,
  - o usando índices
- **Algoritmo para índice primario:**
  - La tabla está ordenada en *A*.
  - para $\sigma_{A \geq V}(r)$ usar el índice para encontrar el primer registro $\geq v$ y escanear la tabla secuencialmente desde allí.
    - El costo se compone de: hi transferencias de bloque, hi accesos a bloque y b transferencias de bloque; b es el número de bloques conteniendo registros con A $\geq v$.
  - para $\sigma_{A \leq V}(r)$ escanear la tabla secuencialmente hasta la primera tupla > *v;* no usar el índice.

Bloque 7: Operadores físicos para selección
Selecciones involucrando comparaciones

Algoritmo para índice secundario para $\sigma_{A \geq V}(r)$ *usando buckets:*

```
Selección_ge_sec(r, R, A, v, f)
  if not exist(f) then create(f, R) fi
  buffer_salida = vacío
  P = lista vacía
  nodo = buscar_hoja_ind(v)
  while nodo ≠ null do // recorre nodos agregando punteros
     for c/entrada (x, pb) de nodo, en orden do // recorre nodo agreg pts
        if x >= v then
           while pb ≠ null do // recorre buckets agregando punteros
              K = leer_bloque(buckets, pb)
              agregar a P todos los punteros de K
              pb = sig(K)
           od
        fi
     od
     if hay hoja siguiente then nodo = leer_bloque(indice, sig(nodo))
     else nodo = null  Fi
  od
  ordenar(P por número de bloque)
  bloque_actual = null
  for p in P do  // accede a registros resultado para punteros de P
     if bloque(p) ≠ bloque_actual then
        B = leer_bloque(r, bloque(p))
        bloque_actual = bloque(p)
     fi
     t = tupla de B en posición pos(p)
     if no_hay_lugar(buffer_salida, t) then
        write(buffer_salida, f) buffer_salida = vacío fi
     insertar(t, buffer_salida)
  od
  if buffer_salida ≠ vacío then
     write(buffer_salida, f) fi
  return f
```

Bloque 7: Operadores físicos para selección Selecciones involucrando comparaciones

```
buscar_hoja_ind(v)
   nodo = leer_bloque(indice, raiz)
   while nodo no es hoja do
        nodo = leer_bloque(indice,
         hijo de nodo que corresponde a v)
   od
   return nodo
```

- **Algoritmo para índice secundario**

- **Resumen**: para $\sigma_{A \geq V}(r)$ usar el índice para encontrar la primera entrada del índice $\geq v$ y escanear el índice secuencialmente desde allí, para encontrar los punteros a los buckets y de ellos a los registros.
- Tenemos *l* cantidad de hojas recorridas, *k* los bloques de buckets leídos y *n* los registros que cumplen A >= v.
- Costo = (hi + (l-1) + k + n) transferencias de bloque y accesos a bloque la misma cantidad.
- Si los resultados se escriben en disco, a esto hay que sumarle:
  - b_f transferencias de bloques y b_f accesos a bloques.

Bloque 7: Operadores físicos para selección Selecciones involucrando comparaciones

- **Algoritmo usando índice secundario en A para $\sigma_{A \leq V}(r)$**
  - para $\sigma_{A \leq V}(r)$ escanear hojas del índice encontrando punteros a los buckets, hasta la primera entrada > *v*. Luego se accede a los registros de la tabla de r usando esos punteros.
  - Hay que retornar los registros que son apuntados desde los buckets.
    - En el peor caso requiere una E/S para cada registro.
- Se puede usar también búsqueda lineal para $\sigma_{A \leq V}(r)$
  - La búsqueda lineal de la tabla puede ser más barata.

Bloque 7: Operadores físicos para selección Selecciones involucrando comparaciones

- Implementación de selecciones complejas:
  - Detallada en Sección 13.3.3 del Silberschatz.
  - Los casos son:
    - Selección de conjunción de predicados básicos.
      - La idea de como proceder se la vio en un ejemplo al estudiar uso de índices.
    - Selección de disyunción de predicados básicos.
      - En caso que se pueda hacer selección de cada predicado básico usando un índice, se usa el índice con el algoritmo correspondiente de selección y luego tomar la unión de los punteros resultantes de cada selección para acceder a los registros de la tabla.
    - Selección de negación: ejercicio de la práctica.

Bloque 8: Producto cartesiano

profe			
legajo	nombres	apellidos	sueldo
p1	"Benjamin"	"Pierce"	3000
p2	"Patricia"	"Selinger"	6000
p3	"Edgar F"	"Codd"	5500
p4	"Barbara"	"Liskov"	5600

curso		
id	legajo	nombre
c1	p2	"Optimización de Consultas"
c2	p3	"Fundamentos de BD"
c3	p2	"Análisis de Datos"
c4	p1	"Fundamentos del Software"
c5	p4	"Programación OO"

–						
p.legajo	nombres	apellidos	sueldo	id	c.legajo	nombre
p1	"Benjamin"	"Pierce"	3000	c1	p2	"Optimización de Consultas"
p1	"Benjamin"	"Pierce"	3000	c2	p3	"Fundamentos de BD"
p1	"Benjamin"	"Pierce"	3000	c3	p2	"Análisis de Datos"
p1	"Benjamin"	"Pierce"	3000	c4	p1	"Fundamentos del Software"
p1	"Benjamin"	"Pierce"	3000	c5	p4	"Programación OO"
p2	"Patricia"	"Selinger"	6000	c1	p2	"Optimización de Consultas"
p2	"Patricia"	"Selinger"	6000	c2	p3	"Fundamentos de BD"
...	...	...	...	...	...	...

Observar el esquema del producto cartesiano en el ejemplo.

¿Qué pueden decir del mismo?

Bloque 8: Producto cartesiano

- Sean los esquemas:

$$r :: (n_1 :: \tau_1, \ldots, n_N :: \tau_N) \qquad s :: (n'_1 :: \tau'_1, \ldots, n'_M :: \tau'_z M)$$

- Entonces el **esquema del producto cartesiano** es:

$$r \times s :: (n_1 :: \tau_1, \ldots, n_N :: \tau_N, n'_1 :: \tau'_1, \ldots, n'_M :: \tau'_M)$$

- Si hay **conflictos de nombres de atributos** se usa el nombre de la tabla para desambiguar.
- Una tabla puede contener **columnas sin nombre**. Para ese caso se usa: _: T
- Voy a poder aplicar el producto cartesiano para tablas donde no todas las columnas tienen nombre.

Bloque 8: Producto cartesiano

- Para definir producto cartesiano podemos inspirarnos en la forma de construir la tabla:

–						
p.legajo	nombres	apellidos	sueldo	id	c.legajo	nombre
p1	"Benjamin"	"Pierce"	3000	c1	p2	"Optimización de Consultas"
p1	"Benjamin"	"Pierce"	3000	c2	p3	"Fundamentos de BD"
p1	"Benjamin"	"Pierce"	3000	c3	p2	"Análisis de Datos"
p1	"Benjamin"	"Pierce"	3000	c4	p1	"Fundamentos del Software"
p1	"Benjamin"	"Pierce"	3000	c5	p4	"Programación OO"
p2	"Patricia"	"Selinger"	6000	c1	p2	"Optimización de Consultas"
p2	"Patricia"	"Selinger"	6000	c2	p3	"Fundamentos de BD"
...	...	...	...	...	...	...

- Junta la primera tupla de profe con cada tupla de curso, junta la segunda tupla de profe con cada tupla de curso y así sigue.
- **¿Cómo se puede generalizar esta idea para el producto cartesiano en general?**

Bloque 8: Producto cartesiano

- Para definir producto cartesiano podemos inspirarnos en la forma de construir la tabla:

–						
p.legajo	nombres	apellidos	sueldo	id	c.legajo	nombre
p1	"Benjamin"	"Pierce"	3000	c1	p2	"Optimización de Consultas"
p1	"Benjamin"	"Pierce"	3000	c2	p3	"Fundamentos de BD"
p1	"Benjamin"	"Pierce"	3000	c3	p2	"Análisis de Datos"
p1	"Benjamin"	"Pierce"	3000	c4	p1	"Fundamentos del Software"
p1	"Benjamin"	"Pierce"	3000	c5	p4	"Programación OO"
p2	"Patricia"	"Selinger"	6000	c1	p2	"Optimización de Consultas"
p2	"Patricia"	"Selinger"	6000	c2	p3	"Fundamentos de BD"
...	...	...	...	...	...	...

- Generalizando:
- t = $v_1 \ldots v_N$ tupla de r
- s tabla
- (estoy definiendo r x s)

v_1	...	v_N	s
v_1	...	v_N	
v_1	...	v_N	
q			

Bloque 8: Producto cartesiano

[] x s = []

(t:r) x s = (juntar t s) ++ r x s

r x s es el q del dibujo

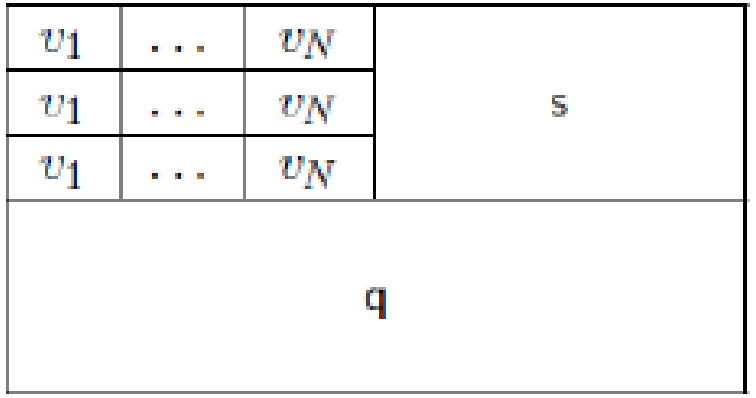
Recordar que concatenar:

```
++ : [a] -> [a] -> [a]

[] ++ l' = l'

(x : l) ++ l' = x : (l ++ l')
```

Ahora definiremos juntar.



Bloque 8: Producto cartesiano
Concatenación de tuplas

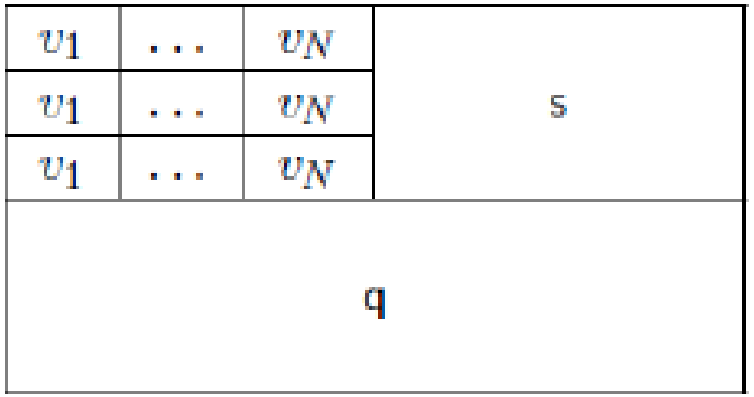
- Definimos la **concatenación de tuplas** como la tupla:

$$(t; t') = (a_1, \ldots, a_N, b_1, \ldots, b_M)$$

- t = a_1,…,a_N, t’ = b_1,…,b_M
- **Usando la concatenación de tuplas, ¿cómo se puede definir juntar?**

Bloque 8: Producto cartesiano
Anexar

- juntar t [] = []
- juntar t (u : s) = (t ; u) : (juntar t s)



Relación entre SQL y Álgegra de Tablas

select $A1,\ldots,An$

from $r1,\ldots,rn$

where P

- Es equivalente a:

$$\Pi_{A1,\ldots,An}(\sigma_p(r_1 \times \ldots \times r_n))$$